Expt - 3

30/10/2026

ACO for Municipal Waste Collection:

Step 1. Initialization

- Set Parameters: Define (the ant count, pheromone importance (α), distance factor (β) and evaporation rate (ρ)
  (to weigh memory, local greediness, & error correction mechanism)
- Set Grid: Initialize all road pheromones to a baseline value (τ_0)
- Input constraint: Loads the road distance, individual point waste amounts & max Truck capacity.

Step 2. Solution Construction (By Ants)

- Start Route: Each ant deploys a truck from the depot location with load = 0.
- Check capacity: Filter unvisited collection points keep only points where current load + point waste $\leq$ truck capacity.
- Move Probabilistically: Choose the next collection point based on road distance (shorter is better) & pheromone strength (stronger is better).
- Emptying pivot: If no remaining collection point fits the truck capacity, return the vehicle to the depot, reset load = 0, & deploy a new vehicle.

Bafna Gold

- Loop - Repeat until all required collection points are serviced

step3: Pheromone Update

- Evaporation - Reduce pheromone levels on all roads to clear out bad choices:
  $\tau \leftarrow (1-\rho)\tau$

- Deposit - Add heavy pheromone rewards to roads used in solutions that achieved the min total travel distance & used the fewer no of vehicles.

step4. output

- Best plan - Return the sequence of routes that satisfies all points with the absolute lowest distance & vehicle count.

o/p.

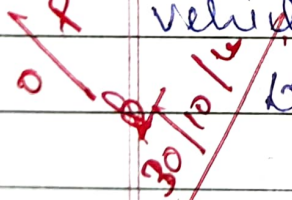
```
== Starting Ant Colony optimization ==
Iteration 01/60 → Best distance: 210.60 km |
        Trucks: 3
Iteration 20/60 → Best distance: 196.30 km |
        Trucks: 3
Iteration 40/60 → Best distance: 196.30 km |
        Trucks: 3
Iteration 60/60 → Best distance: 196.30 km |
        Trucks: 3
```

Process terminated after 30 steps

== Optimal Municipal waste collection plan ==

- Minimum distance : 196.30 km
- Minimum Vehicles (Trucks) Required : 3

Vehicle #1 : Path : [0 → 5 → 1 → 0] |
Loaded weight : 8 / 8 Tons
Vehicle #2 : Path : [0 → 2 → 3 → 0] |
Loaded weight : 8 / 8 Tons
Vehicle #3 : Path : [0 → 4 → 0] |
Loaded weight : 2 / 8 Tons.



```python
import random

# Input
n = int(input("Enter number of collection points: "))
capacity = int(input("Enter truck capacity: "))

demand = list(map(int, input("Enter waste at each point: ").split()))
distance = list(map(int, input("Enter distance from depot to each point: ").split()))

pheromone = [1.0] * n

alpha = 1
beta = 2
rho = 0.5
Q = 100

best_distance = float('inf')
best_route = []

# ACO
for iteration in range(1, 61):

    # Evaporation
    for i in range(n):
```

```
# Evaporation
for i in range(n):
    pheromone[i] *= (1 - rho)

route = []
visited = []
load = 0
total_distance = 0
trucks = 1

while len(visited) < n:

    choices = []

    for i in range(n):
        if i not in visited and load + demand[i] <= capacity:
            p = pheromone[i] ** alpha * (1 / distance[i]) ** beta
            choices.append((i, p))

    if not choices:
        trucks += 1
        load = 0
        continue
```

```
    for i in range(n):
        if i not in visited and load + demand[i] <= capacity:
            p = pheromone[i] ** alpha * (1 / distance[i]) ** beta
            choices.append((i, p))

    if not choices:
        trucks += 1
        load = 0
        continue

    points = [x[0] for x in choices]
    weights = [x[1] for x in choices]

    next_point = random.choices(points, weights=weights)[0]

    route.append(next_point + 1)
    visited.append(next_point)

    load += demand[next_point]
    total_distance += 2 * distance[next_point]

# Deposit
for i in visited:
    pheromone[i] += Q / total_distance
```

```
    # Deposit
    for i in visited:
        pheromone[i] += Q / total_distance

    if total_distance < best_distance:
        best_distance = total_distance
        best_route = route[:]
        best_trucks = trucks

    if iteration % 20 == 0:
        print("Iteration:", iteration,
              "Best Distance:", best_distance,
              "Trucks:", best_trucks)

# Output
print("\nBest Route:", best_route)
print("Best Distance:", best_distance)
print("Number of Trucks:", best_trucks)
```

```
Enter number of collection points: 4
Enter truck capacity: 10
Enter waste at each point: 4 5 3 6
Enter distance from depot to each point: 10 15 20 12
Iteration: 20 Best Distance: 114 Trucks: 2
Iteration: 40 Best Distance: 114 Trucks: 2
Iteration: 60 Best Distance: 114 Trucks: 2
```